

MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

1. Setup: Employees Table

```
CREATE DATABASE IF NOT EXISTS employeeedb1;  
USE employeeedb1;
```

```
CREATE TABLE IF NOT EXISTS employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees VALUES  
(101,'Rahul',1,50000),  
(102,'Priya',2,65000),  
(103,'Anil',1,55000),  
(104,'Sneha',3,70000),  
(105,'Kiran',2,48000);
```

2. Verify Data

Question: Display all employee records.

```
80 • SELECT * FROM employees;
```

Result Grid

Filter Rows:

Edit:

Export

	EmpID	EmpName	Salary	DeptID	ManagerID	Email
▶	101	Rahul	60000.00	1	NULL	rahul@gmail.com
	102	Anjali	65000.00	2	101	anjali@gmail.com
	103	Rohan	49500.00	1	101	rohan@gmail.com
	105	Kiran	77000.00	1	101	kiran@gmail.com
	106	Sneha	62000.00	4	105	sneha@gmail.com
	107	Arjun	32000.00	NULL	101	arjun@gmail.com
	108	Vijay	58000.00	2	102	vijay@gmail.com
*	NULL	NULL	NULL	NULL	NULL	NULL

Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees.

```
1 DELIMITER $$
2 • CREATE PROCEDURE GetEmployees()
3 BEGIN
4     SELECT * FROM Employees;
5 END $$
6 DELIMITER ;
7 • CALL GetEmployees();
```

Result Grid						
Filter Rows:						
Export:  Wrap Cell Content						
	EmpID	EmpName	Salary	DeptID	ManagerID	Email
▶	101	Rahul	60000.00	1	NULL	rahul@gmail.com
	102	Anjali	65000.00	2	101	anjali@gmail.com
	103	Rohan	49500.00	1	101	rohan@gmail.com
	105	Kiran	77000.00	1	101	kiran@gmail.com
	106	Sneha	62000.00	4	105	sneha@gmail.com
	107	Arjun	32000.00	NULL	101	arjun@gmail.com
	108	Vijay	58000.00	2	102	vijay@gmail.com

Procedure 2 – IN Parameter

Question: Create a procedure that accepts an employee ID and displays that employee's details.

```

3 • DROP PROCEDURE IF EXISTS GetEmployeeById;
4 DELIMITER $$
5
6 • CREATE PROCEDURE GetEmployeeById(IN eid INT)
7 BEGIN
8     SELECT * FROM employees
9     WHERE EmpId = eid;
10 END $$
11
12 DELIMITER ;
13
14 • CALL GetEmployeeById(102);
15

```

Result Grid		Filter Rows:		Export:		Wrap Cell Content:
	EmpID	EmpName	Salary	DeptID	ManagerID	Email
	102	Anjali	65000.00	2	101	anjali@gmail.com

Procedure 3 – Update Salary

Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.

```

12 DELIMITER ;
13
14 • CALL GetEmployeeById(102);
15 • DROP PROCEDURE IF EXISTS IncreaseSalary;
16
17 DELIMITER $$
18 • CREATE PROCEDURE IncreaseSalary(
19     IN eid INT,
20     IN amount DECIMAL(10,2)
21 )
22 BEGIN
23     UPDATE employees
24     SET Salary = Salary + amount
25     WHERE EmpId = eid;
26 END $$
27
28 DELIMITER ;
29
30 • CALL IncreaseSalary(101,5000);
31 • SELECT * FROM employees WHERE EmpId=101;
32

```

Result Grid

Filter Rows:

Edit

Export

	EmpID	EmpName	Salary	DeptID	ManagerID	Email
▶	101	Rahul	75000.00	1	NULL	rahul@gmail.com
	NULL	NULL	NULL	NULL	NULL	NULL

Procedure 4 – Aggregate Function

Question: Create a procedure to calculate the average salary of all employees.

```

33 DELIMITER $$
34
35 CREATE PROCEDURE AvgSalary()
36 BEGIN
37     SELECT AVG(Salary) AS AverageSalary
38     FROM Employees;
39 END $$
40
41 DELIMITER ;
42
43 CALL AvgSalary();
44

```

Result Grid	Filter Rows:	Export:	Wrap
AverageSalary			
59785.714286			

Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.

```

35 INSERT IGNORE INTO Departments VALUES
36 (1, 'CSE'),
37 (2, 'ECE'),
38 (3, 'EEE');
39
40 DROP PROCEDURE IF EXISTS EmployeeDepartment;
41
42 DELIMITER $$
43
44 CREATE PROCEDURE EmployeeDepartment()
45 BEGIN
46     SELECT
47         e.EmpName,
48         d.DeptName,
49         e.Salary
50     FROM Employees e
51     JOIN Departments d
52     ON e.DeptID = d.DeptID;
53 END $$
54
55 DELIMITER ;
56
57 CALL EmployeeDepartment();
58

```

Result Grid	Filter Rows:	Export:	Wrap Cell Conte
EmpName	DeptName	Salary	
Rahul	CSE	75000.00	
Rohan	CSE	49500.00	
Kiran	CSE	77000.00	
Anjali	ECE	65000.00	
Vijay	ECE	58000.00	
Sneha	MBA	62000.00	

Procedure 6 – GROUP BY and AVG

Question: Create a procedure to display department-wise employee count and average salary.

```

1 DELIMITER $$
2
3 CREATE PROCEDURE DepartmentAverageSalary()
4 BEGIN
5     SELECT
6         d.DeptName,
7         COUNT(e.EmpID) AS EmployeeCount,
8         AVG(e.Salary) AS AverageSalary
9     FROM Employees e
10    JOIN Departments d
11      ON e.DeptID = d.DeptID
12   GROUP BY d.DeptID, d.DeptName;
13 END $$
14
15 DELIMITER ;
16
17 CALL DepartmentAverageSalary();
18

```

DeptName	EmployeeCount	AverageSalary
CSE	3	67166.666667
ECE	2	61500.000000
MBA	1	62000.000000

Procedure 7 – IF...ELSE

Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.

```

1 DELIMITER $$
2
3 CREATE PROCEDURE SalaryStatus(IN eid INT)
4 BEGIN
5     DECLARE sal DECIMAL(10,2);
6
7     SELECT Salary INTO sal
8     FROM Employees
9     WHERE EmpID = eid;
10
11     IF sal >= 60000 THEN
12         SELECT 'High Salary' AS Status;
13     ELSE
14         SELECT 'Low Salary' AS Status;
15     END IF;
16 END $$
17 DELIMITER ;
18 CALL SalaryStatus(102);
19

```

Status
High Salary

Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.

The screenshot displays a SQL script in a text editor with line numbers 1 through 14. The script defines a stored procedure named `EmployeeCount` that takes an output parameter `total` of type `INT`. The procedure's logic is to select the count of all rows in the `Employees` table and store it in the `total` variable. After the procedure is created, it is called with `@total` as the argument. Finally, a `SELECT` statement is used to retrieve the value of `@total`, which is displayed as `TotalEmployees` in the result grid below the script editor.

```
1 DELIMITER $$
2
3 CREATE PROCEDURE EmployeeCount(OUT total INT)
4 BEGIN
5     SELECT COUNT(*) INTO total
6     FROM Employees;
7 END $$
8
9 DELIMITER ;
10
11 CALL EmployeeCount(@total);
12
13 SELECT @total AS TotalEmployees;
14
```

Below the script editor, the **Result Grid** shows the output of the last query:

TotalEmployees
7

Procedure 9 – INOUT Parameter

Question: Receive a bonus through an INOUT parameter and add 5000 to it.

```

1 DELIMITER $$
2 CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
3 BEGIN
4     SET bonus = bonus + 5000;
5 END $$
6 DELIMITER ;
7 SET @bonus = 10000;
8 CALL AddBonus(@bonus);
9 SELECT @bonus AS UpdatedBonus;
10

```

Result Grid

UpdatedBonus
15000.00

Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.

```

1 DELIMITER $$
2 CREATE PROCEDURE TransferSalaryAmount(
3     IN fromEmp INT,
4     IN toEmp INT,
5     IN amount DECIMAL(10,2)
6 )
7 BEGIN
8     START TRANSACTION;
9     UPDATE employees
10    SET salary=salary-amount
11    WHERE emp_id=fromEmp;
12    UPDATE employees
13    SET salary=salary+amount
14    WHERE emp_id=toEmp;
15    COMMIT;
16 END $$
17 DELIMITER ;
18 CALL TransferSalaryAmount(101,102,2000);
19 SELECT * FROM employees WHERE emp_id IN (101,102);
20

```